

Nexus Control

Cloud-Native IT Support & Incident Management Platform

AWS Account ID	473009222991
AWS Region	ap-south-1 (Mumbai)
Date	August 2026
Team Members	Nikhil Jallepalli Rishitha Kothuri Vinay Kalla
Evaluator / Director	Ramya Uppala

Table of Contents

1. Project Overview
2. Architecture Overview
3. AWS Services Used
4. Application Flow
5. Network Architecture
6. Terraform Structure
7. Docker & Container Approach
8. Database Configuration
9. Secrets Management
10. Frontend Deployment
11. AWS S3 Pre-Signed Upload & Lambda Flow
12. CI/CD Architecture
13. CloudWatch & Monitoring
14. Security Implementation
15. AWS Monthly Cost Estimate
16. Testing Results
17. Step-by-Step Deployment
18. Terraform Destroy Evidence
19. Problems Encountered & Root-Cause Solutions
20. Individual Contribution Mapping
- Appendix A. Technology Stack Summary
- Appendix B. Glossary of Terms
- Appendix C. Recommendations for Production Readiness

1. Project Overview

Nexus Control is a cloud-native, enterprise-grade IT Support & Incident Management platform designed to streamline IT support requests, automate SLA compliance tracking, and eliminate server network overhead through direct Amazon S3 pre-signed file uploads.

The platform was built as a proof of concept to demonstrate that a small team can design, provision, and operate a production-style, multi-tier AWS deployment end to end — from infrastructure-as-code and container orchestration through to a polished, role-aware frontend — while keeping the running cost and operational footprint minimal. The sections that follow document the architecture, the AWS services used, the application and network design, the CI/CD and deployment process, the security model, and the issues the team encountered and resolved along the way.

Core Capabilities

- **Role-Isolated Security** — granular boundaries between System Administrators (ROLE_ADMIN), IT Support Engineers (ROLE_SUPPORT_ENGINEER), and Employees (ROLE_EMPLOYEE). Employees are strictly isolated to viewing tickets they personally created.
- **Direct AWS S3 Attachment Pipeline** — bypasses backend server memory/CPU by issuing secure, 15-minute time-limited AWS SDK v2 pre-signed URLs directly to client browsers.
- **Real-Time Analytics & SLA Engine** — automated SLA target tracking (Urgent: 2h, High: 8h, Medium: 24h, Low: 72h) with interactive dashboard visualizations.
- **Zero-Downtime CI/CD** — automated GitHub Actions pipeline validating Terraform IaC formatting, executing JUnit 5 test suites, building optimized React bundles, and deploying container revisions to AWS ECS Fargate cluster.

Objectives of the Proof of Concept

- **Validate** that a role-based ticketing workflow can be delivered on a serverless/managed AWS stack without a dedicated operations team.
- **Demonstrate** infrastructure reproducibility: the entire environment can be created and torn down on demand using Terraform, keeping idle cost close to zero.
- **Prove** that file attachments can bypass the application server entirely, reducing backend compute and memory pressure under load.
- **Establish** a repeatable CI/CD pipeline so that future feature work ships through an automated, tested path rather than manual deployment.

Intended Audience

This document is written for the evaluation panel reviewing the proof of concept, as well as for engineers who may extend or operate the platform afterward. It is organized to move from the high-level project summary and architecture down to specific configuration, deployment steps, and individual contribution mapping, so that a reader can either skim the overview sections or drill into implementation detail as needed.

2. Architecture Overview

The Nexus Control architecture is deployed across a Multi-AZ Virtual Private Cloud (VPC) in AWS ap-south-1, comprising public, private, and database subnet tiers.

The design follows a conventional three-tier separation of concerns: a public-facing edge (load balancer), a private application tier that is never directly reachable from the internet, and an isolated data tier that only the

application tier can reach. This layered approach limits the blast radius of any single compromised component and keeps the database completely unreachable from outside the VPC.

Tier Summary

Tier	Component / Resource ID	Description
Frontend	ticketdesk-frontend-0ee57288	AWS S3 static website hosting (holds compiled React UI)
Entry & Routing	ticketdesk-m1-alb	AWS Application Load Balancer, listening on port 80
Application	ECS Fargate backend tasks	Spring Boot backend containers running in private subnets on port 8080
Database	ticketdesk-db	AWS RDS MySQL 8.0 instance in private database subnets on port 3306
Storage	ticketdesk-uploads-*	AWS S3 bucket accepting direct pre-signed client uploads
Secrets & Security	AWS Secrets Manager / SSM	Stores database credentials and parameter configurations

3. AWS Services Used

Every managed service in the stack was chosen to minimize operational burden: there are no self-managed servers to patch, and each service scales or fails over automatically within the limits of its configured tier. The table below lists each AWS service, the specific resource created for Nexus Control, and the role it plays.

Service	Resource	Purpose
Amazon VPC	ticketdesk-vpc	Multi-AZ network isolation across 4 subnets
Amazon ECS Fargate	ticketdesk-m1-cluster	Serverless container execution for the Spring Boot REST backend (0.25 vCPU, 0.5 GB RAM)
AWS Application Load Balancer	ticketdesk-m1-alb	High-availability HTTP entry point distributing traffic to private ECS tasks
Amazon S3 — Static Website	ticketdesk-frontend-0ee57288	Static hosting for the compiled React production frontend
Amazon S3 — Attachments	ticketdesk-uploads-*	Direct pre-signed bucket for logs, screenshots, and PDFs

Amazon RDS MySQL 8.0	ticketdesk-db	Managed relational database storing users, roles, tickets, attachments, and comments
AWS Secrets Manager	ticketdesk-db-password	Secure storage and injection of database password
AWS Systems Manager (SSM)	/ticketdesk/* parameters	Parameter Store configuration repository (DB URL, username, regions, buckets)
Amazon ECR	ticketdesk-backend	Docker container registry storing versioned backend images
Amazon CloudWatch Logs	/ecs/ticketdesk-backend	Centralized container log aggregation and operational diagnostics

Why This Service Mix

The stack deliberately favors managed and serverless services over self-hosted alternatives. ECS Fargate removes the need to patch or size EC2 instances for the backend; RDS Multi-AZ capability removes the need to manage database replication and failover manually; and S3 static website hosting removes the need for a dedicated web server for the frontend entirely. This keeps the number of components the team is responsible for operating to a minimum, which matters for a small team running a proof of concept alongside other responsibilities.

4. Application Flow

The sequence below traces a typical user session from authentication through ticket creation and file attachment, illustrating how responsibility is split between the frontend, the backend, and AWS-managed storage.

1. User logs in via the React frontend and receives a stateless JWT token.
2. User creates a ticket; the React frontend calls POST /api/tickets via the ALB to ECS Fargate.
3. Spring Boot validates the JWT, persists the ticket into RDS MySQL, and returns the created DTO.
4. User attaches a diagnostic screenshot; the frontend requests a pre-signed URL via /api/attachments/presigned-put.
5. Spring Boot backend issues a 15-minute S3 pre-signed PUT URL using the AWS SDK v2 S3Presigner.
6. The browser streams the file payload directly to the Amazon S3 uploads bucket, bypassing backend bandwidth entirely.
7. S3 fires an s3:ObjectCreated:* notification triggering the Python Lambda thumbnail function, which resizes and writes a thumbnail back to the bucket.

Design Rationale

Two design choices shape this flow. First, authentication is stateless: the backend never stores session state, so any ECS task can serve any request, which keeps horizontal scaling simple and removes the need for sticky sessions at the load balancer. Second, file upload traffic never touches the application tier — the backend only issues a short-lived, single-use URL, and the actual bytes travel straight from the browser to S3. This keeps large attachments (screenshots, logs, PDFs) from consuming ECS task memory or CPU, so the same Fargate task can comfortably serve API traffic regardless of how many attachments are being uploaded concurrently.

5. Network Architecture

The VPC CIDR block is configured as 10.0.0.0/16 across ap-south-1a and ap-south-1b. The network is segmented into public and private subnet tiers to enforce network-level isolation.

Subnet Tier	CIDR Blocks	Hosts
Public	10.0.1.0/24, 10.0.2.0/24	Internet Gateway, NAT Gateways & Application Load Balancer
Private (App/DB)	10.0.10.0/24, 10.0.11.0/24	ECS Fargate containers & RDS MySQL Database endpoints

Security groups enforce strict ingress rules:

- **ALB Security Group:** accepts inbound HTTP traffic on port 80 from any public source (0.0.0.0/0).
- **ECS Security Group:** allows inbound traffic on port 8080 ONLY if it originates from the ALB Security Group.
- **RDS Security Group:** allows inbound database queries on port 3306 ONLY if they originate from the ECS Security Group.

Defense in Depth

Each security group is scoped to the minimum source it needs to function, rather than to a broad CIDR range. Because the ECS security group only accepts port 8080 from the ALB's security group (not from any IP in the private subnet), a compromised host elsewhere in the private tier still could not reach the backend directly. The same principle applies to RDS: only the ECS security group is permitted on port 3306, so the database is unreachable even from other resources inside the VPC unless they are explicitly granted access.

6. Terraform Structure

The infrastructure is provisioned declaratively via Terraform files located inside the /terraform directory. The code is structured as follows:

- **provider.tf** — AWS provider declaration, regions, and default tagging rules.
- **vpc.tf** — VPC, subnet divisions (public and private), route tables, internet and NAT gateways.
- **security_groups.tf** — Firewall rules mapping ingress and egress for ALB, ECS, and RDS.
- **alb.tf** — ALB creation, HTTP listeners, target groups, and path-based routing rules.
- **ecs.tf** — ECS Cluster, Fargate task definition, ECR repository mapping, and Service definitions.
- **rds.tf** — RDS MySQL instance parameters, DB subnet group, and SSM parameter mappings.
- **s3.tf** — Static website hosting bucket and attachment S3 bucket configurations with CORS configurations.
- **lambda.tf** — Lambda archive packaging, thumbnail function deployment, triggers, and invocation policies.
- **iam.tf** — ECS execution, ECS task, and Lambda execution IAM roles with custom least-privilege policies.
- **monitoring.tf** — CloudWatch log groups registration and CPU/Memory utilization alarm thresholds.

Why Modularize

Splitting the infrastructure into discrete concern areas keeps each AWS concern independently testable and reviewable: a change to the S3 CORS policy, for example, touches only s3.tf and does not risk an unrelated diff in

the ECS task definition. It also means the configurations can be parameterized to stand up staging or production environments without duplicating logic. Terraform state is tracked centrally so that terraform plan shows a complete, accurate diff of any change before it is applied, and terraform destroy can tear down the entire environment cleanly.

7. Docker & Container Approach

The backend is containerized using a multi-stage Dockerfile located at backend/Dockerfile. The compilation stage uses an eclipse-temurin JDK 25 on Alpine to build the Maven package, and the runtime stage copies the resulting jar into a minimal eclipse-temurin JRE 25 Alpine container. This separation keeps the final image lightweight, reduces the security attack surface, and speeds up ECR pushes. The container runs under a custom non-root system group and user ('appuser'/'appgroup') to enforce least-privilege security at runtime. The Spring Boot Actuator endpoint /health is configured as the container health check, allowing ECS to auto-replace unhealthy container tasks.

8. Database Configuration

The database tier uses a managed Amazon RDS MySQL 8.0 instance running on a db.t3.micro burstable instance class to remain Free-Tier eligible. It is isolated in private database subnets with no public IP address. Spring Boot connects to RDS via HikariCP connection pooling configured with a maximum size of 10 connections. flyway-style schema migrations are executed automatically at boot, running Flyway DDL updates dynamically from the application classpath. This ensures that schema modifications are version-controlled alongside application code, making database setup fully automated in fresh environments.

9. Secrets Management

Secrets are managed dynamically to prevent hardcoding in the codebase: non-sensitive configurations (such as JDBC URLs, S3 bucket names, and regions) are stored in AWS Systems Manager (SSM) Parameter Store. Sensitive data, specifically the MySQL database password, is stored inside AWS Secrets Manager. When ECS Fargate boots a task, the ECS agent fetches these parameters and decrypts the database secret under the task's Execution IAM role, injecting them directly as OS environment variables (DB_URL, DB_USER, DB_PASSWORD) into the Spring Boot container. This keeps credentials out of the Git repository, the compiled Docker image, and the Terraform code files.

10. Frontend Deployment

The React 18 frontend is compiled using Vite into static bundles (HTML, JS, and CSS) and deployed using S3 static website hosting (ticketdesk-frontend-0ee57288) with public read policies. In the production CDN model, a CloudFront distribution sits in front of S3, caching assets at edge locations and routing API requests (/api/*) to the ALB. In the POC single-artifact mode, built frontend static resources are copied directly into the Spring Boot backend/src/main/resources/static/ directory. When packaged, the Spring Boot application jar serves both the frontend assets and backend API, allowing the entire application to run through the Application Load Balancer (ALB) on port 80/8080.

11. AWS S3 Pre-Signed Upload & Lambda Flow

To handle file attachments (such as screenshots, PDFs, and log files) without exhausting backend bandwidth and CPU resources, a secure S3 Pre-Signed Upload flow is implemented: first, the frontend requests an upload URL from the backend; the backend controller calls S3Presigner to generate a cryptographically signed, 15-minute time-limited PUT URL; the browser uploads the file directly to the S3 uploads bucket using this URL. Once the upload finishes, S3 triggers an s3:ObjectCreated:* notification that invokes the Python 3.9 Lambda function

('ticketdesk-thumbnail-generator'). The Lambda handler uses its execution role to read the image, resizes it, and saves a scaled-down thumbnail version back to the bucket under a 'thumbnails/' prefix. The handler checks for the 'thumbnails/' key prefix at start to prevent recursive loop invocation.

12. CI/CD Architecture

The CI/CD pipeline is fully automated using GitHub Actions defined in `.github/workflows/ci.yml`. On every commit or pull request to the master/main branches, the workflow executes a sequence of stages: (1) Check out code, (2) Set up Java 25.0.1 (Temurin) and Node.js 26.2.0, (3) Run backend Maven JUnit unit tests, (4) Install frontend dependencies, run linting, and compile React static bundles, copying them to the Spring Boot resources directory, (5) Scan for committed secrets using Trufflehog, (6) Authenticate to AWS, log in to ECR, and build the Docker image tagged with the git commit SHA, pushing it to ECR, (7) Download the active task definition, update the image tag, and register it to deploy to ECS Fargate with stability checks, and (8) Retrieve the ALB DNS URL and run dynamic smoke tests (Create Ticket, Fetch Tickets, Dashboard stats) to verify integration health.

13. CloudWatch & Monitoring

Unified observability is achieved by aggregating container stdout and stderr streams into the `/ecs/ticketdesk-backend` CloudWatch Log Group. Standard logs include HikariCP connection status, Flyway migrations, Hibernate SQL queries, and Spring Boot exceptions. Container CPU and memory utilization metrics are monitored, and custom CloudWatch Alarms are configured to trigger if task resource consumption exceeds 85% for two consecutive 5-minute evaluation periods. Spring Boot Actuator endpoints (`/actuator/health`) expose health indicators for database connectivity, disk space, and memory pools to ALB health check target groups.

14. Security Implementation

Security is enforced at multiple layers rather than relying on a single control point. Network-level isolation restricts database access solely to containers. Application-level security is handled by Spring Security 6, verifying stateless JWT signatures and resolving user roles (`ROLE_ADMIN`, `ROLE_SUPPORT_ENGINEER`, `ROLE_EMPLOYEE`) for access authorization. The database password is encrypted in Secrets Manager. Ephemeral S3 pre-signed URLs restrict file upload access. In the database, repository queries are parameter-bound and scoped strictly by the authenticated user's ID, ensuring employee ticket isolation.

Layered Security Model

- **Identity & Auth:** Stateless JWT authentication (HMAC-SHA256) and BCrypt password hashing (strength 10).
- **Access Control:** Spring Security 6 role-based path protection and repository-level user-scoped query isolation.
- **Network Firewall:** Unidirectional security group flows (ALB -> ECS SG -> RDS SG) and NAT Gateway egress routing.
- **IAM Profiles:** Least-privilege execution roles for container startup, runtime task roles, and Lambda execution.

15. AWS Monthly Cost Estimate

Estimated monthly expenses for the Nexus Control POC, assuming a development environment scale and Free-Tier eligibility:

Resource	Estimated Monthly Cost (USD)
----------	------------------------------

ECS Fargate task (0.25 vCPU, 0.5 GB RAM)	\$9.01
Application Load Balancer	\$16.42
RDS MySQL Database (db.t3.micro)	\$12.41
S3 Storage & CloudWatch Logs	\$0.60
Total Estimated Monthly Cost	\$38.44

Cost Control Approach

To minimize billing in the POC environment, resources are right-sized (burstable db.t3.micro database instance, minimal Fargate task sizes, and minimal S3/CloudWatch usage). To eliminate idle cost completely, the environment is destroyed via Terraform when not actively undergoing testing.

16. Testing Results

All stages of verification pass successfully in the CI/CD pipeline: backend JUnit tests run cleanly, TypeScript linter checks and Vite compilation audits complete without warnings, Trufflehog verifies zero hardcoded keys, and E2E smoke tests successfully perform automated Create, Query, and Dashboard stats requests against the ALB DNS endpoint on deploy.

17. Step-by-Step Deployment

Deploying the Nexus Control infrastructure is automated via the following steps:

1. Navigate to /terraform directory and run 'terraform init' to download provider plugins.
2. Execute 'terraform apply -auto-approve' to provision the VPC network, database, container tasks, load balancer, S3, and Lambda.
3. Push local commits to the main/master branch on GitHub to trigger ECR builds and ECS service updates via Actions.
4. Retrieve the ALB endpoint DNS and verify the health check endpoint using a web browser.

Repeatability

Because the complete infrastructure is defined as code and deployments are automated through GitHub Actions, the environment can be rebuilt from scratch in minutes without manual Console steps. This was verified by running multiple tear-down and rebuild cycles during testing.

18. Terraform Destroy Evidence

Clean deletion of all 32 AWS-provisioned infrastructure resources is achieved using terraform destroy. Sample execution logs verifying complete resource destruction:

```
aws_db_instance.db: Destroying... [id=ticketdesk-db]
aws_ecs_service.backend_service: Destroying...
[id=arn:aws:ecs:ap-south-1:473009222991:service/ticketdesk-m1-service]
...
aws_vpc.main: Destroying... [id=vpc-0be1f9570a43a53a9]
aws_vpc.main: Destruction complete after 3s
```

Destroy complete! Resources: 32 destroyed.

19. Problems Encountered & Root-Cause Solutions

The issues surfaced during testing are documented below with their root-causes and applied resolutions:

Problem	Root Cause / Resolution
RDS Target Connection Timeouts	The private RDS MySQL instance blocked connection requests from ECS Fargate container tasks. Resolution: Added a security group rule allowing inbound TCP port 3306 traffic from ticketdesk-m1-task-sg.
Browser CORS Policy Blockages	React AJAX requests failed cross-origin audits due to separate domains for S3 web hosting and the ALB endpoint. Resolution: Unified endpoint routing under the ALB. S3 static assets are copied to Spring Boot resources to serve both under the same port/domain.
Secrets Manager Access Denied	ECS tasks crashed during initialization due to missing AWS permission mappings. Resolution: Added secretsmanager:GetSecretValue and ssm:GetParameters permissions to the ticketdesk-m1-execution-role.

Lessons Carried Forward

Configuration issues must be declared as code in Terraform and GitHub Actions workflows, rather than patched manually in the AWS Console. Declaring security group mappings, S3 CORS rules, and execution IAM roles in code ensures the entire stack remains fully reproducible and prevents configuration drift.

20. Individual Contribution Mapping

The core responsibilities and contributions implemented by the team members are mapped below:

Member	Role	Evaluation Area	Module / Files	Contribution Summary
Nikhil Jallepalli	Cloud & Infrastructure	Networking, Container Orchestration, CI/CD	VPC configs, alb.tf, ecs.tf, .github/workflows/ci.yml, frontend/src/App.tsx	Configured the Multi-AZ VPC network, routing tables, and security groups. Deployed ALB listeners and ECS Fargate services. Built the Github Actions CI/CD pipeline, and integrated React telemetry HUD components showing live latency metrics.

Rishitha	Lead Developer	Backend Logic, UI Components, QA Testing	backend/src/main/java/, frontend/src/App.tsx, tests/	Implemented Spring Boot REST controllers and repository classes for incident lifecycle, comments, and user records. Developed frontend React router pages, CSS styles, and backend JUnit 5 verification suites.
Vinay	DevOps & Database Specialist	S3 Attachments, RDS Database, Observability	s3.tf, rds.tf, monitoring.tf, AttachmentServiceImpl.java	Provisioned RDS MySQL instance and DB subnet groupings. Configured S3 attachments bucket, CORS policies, and S3 pre-signed upload routines. Built CloudWatch logs integration and Memory/CPU metrics thresholds.

Summary

Across the three contributors, the team covered infrastructure and DevOps, backend security and data, storage and integration, frontend delivery, and minor backend support — the core disciplines needed to take a proof of concept from design through a working, deployed, and repeatably provisioned system. The sections above document that system in the order a reviewer would typically want to evaluate it: what it does, how it is architected, how it is built and deployed, how it is secured, what it costs to run, and how the work was distributed across the team.

Appendix A. Technology Stack Summary

Layer	Technology	Notes
Frontend framework	React 18	Compiled with Vite; deployed as a static bundle
Frontend charts	Recharts	Interactive SLA and analytics visualizations
Backend framework	Spring Boot	REST API, runs on Java 25 (eclipse-temurin)
Security	Spring Security	Stateless JWT (HMAC-SHA256), BCrypt password hashing
Database	MySQL 8.0	AWS RDS instance, db.t3.micro
Containerization	Docker	Adoptopenjdk/openjdk25 JRE runtime image
Container orchestration	Amazon ECS Fargate	Serverless container execution, no EC2 virtual machines to manage
Infrastructure as Code	Terraform (HCL)	Modularized code structures in /terraform
CI/CD	GitHub Actions	Build, test, ECR container push, and ECS update-service pipeline (.github/workflows/ci.yml)
Object storage	Amazon S3	Static website hosting and pre-signed attachment uploads

Together, this stack was selected to keep every layer either fully managed by AWS or fully defined as code, so that the proof of concept could be reproduced, evaluated, and torn down repeatably with minimal manual intervention.

Appendix B. Glossary of Terms

- **ALB (Application Load Balancer)** — an AWS-managed layer 7 load balancer that distributes incoming HTTP traffic across backend targets.
- **CIDR block** — a notation (e.g. 10.0.0.0/16) describing a range of IP addresses assigned to a network or subnet.
- **CORS (Cross-Origin Resource Sharing)** — a browser security mechanism that restricts which origins may make requests to a given resource, configured here on the S3 uploads bucket.
- **ECR (Elastic Container Registry)** — AWS's managed Docker image registry, used here to store versioned backend images.
- **ECS Fargate** — a serverless compute engine for containers that removes the need to provision or manage the underlying EC2 instances.

- **Flyway** — a database migration tool that applies versioned SQL scripts automatically, keeping schema changes tracked alongside application code.
- **HMAC-SHA256** — a cryptographic algorithm used here to sign and verify JWTs, ensuring a token cannot be forged or altered without detection.
- **IaC (Infrastructure as Code)** — defining cloud infrastructure in declarative configuration files (Terraform, in this project) rather than provisioning resources manually.
- **JWT (JSON Web Token)** — a compact, signed token used to represent an authenticated user's identity and claims without requiring server-side session storage.
- **Multi-AZ** — a deployment pattern that replicates a resource across two or more AWS Availability Zones for high availability.
- **Pre-signed URL** — a time-limited, cryptographically signed URL that grants temporary access to a specific S3 object without exposing AWS credentials.
- **RBAC (Role-Based Access Control)** — an authorization model that grants permissions based on a user's assigned role rather than per-user rules.
- **VPC (Virtual Private Cloud)** — an isolated, logically separated network within AWS in which resources such as ECS tasks and RDS instances are deployed.

Appendix C. Recommendations for Production Readiness

The current deployment is deliberately scoped as a proof of concept. The items below outline what the team would prioritize before treating this as a production system, based on the gaps observed during the evaluation period.

- **HTTPS termination** — the ALB currently listens on port 80 only; a production deployment would add an ACM certificate and an HTTPS listener, redirecting all HTTP traffic to HTTPS.
- **Auto Scaling** — the ECS service currently runs a fixed task count; adding an Application Auto Scaling policy tied to CPU/memory or request count would let the service handle variable load without manual intervention.
- **CloudWatch Alarms** — the Actuator health and metrics endpoints already exist (Section 13); the next step is wiring them into CloudWatch Alarms and a dashboard so the team is notified proactively rather than discovering issues by checking logs.
- **CDN distribution** — fronting S3 static hosting with Amazon CloudFront would reduce frontend latency for geographically distributed users and allow HTTPS on the frontend origin as well.
- **Backup and retention policy** — formalizing RDS automated backup retention and S3 lifecycle rules for older attachments would round out the operational posture beyond what a proof of concept requires.